



Penetration Testing

Umbrae (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	6
Security Rating	7
Severity Definitions	8
Status Definitions	9
Penetration Test Findings	10
Conclusion	24
Our Methodology	25
Disclaimers	27
About Hashlock	28

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Umbrae team partnered with Hashlock to conduct a security Penetration Test of their Umbrae Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

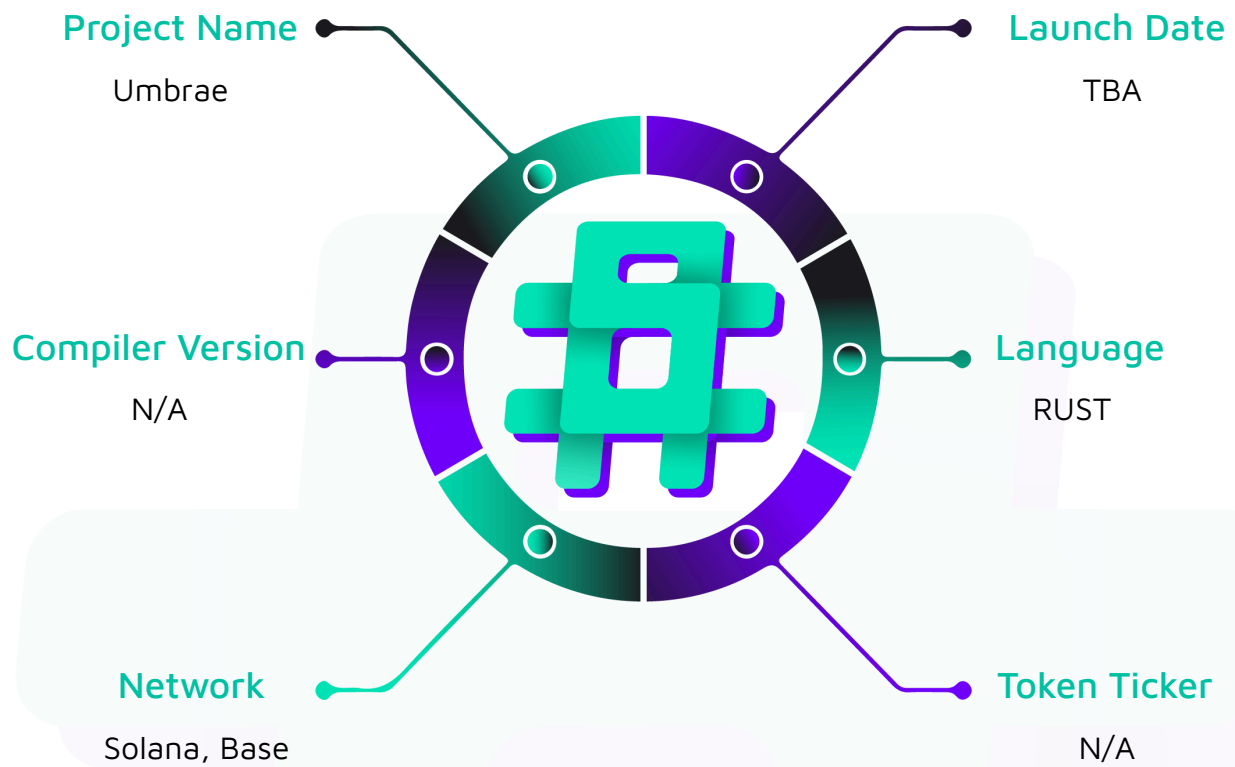
Umbrae is launching the first multi-chain DLMM on Base Network with the lowest operation fees in DeFi. The platform delivers 97-99% transaction success rates through proprietary infrastructure with ongoing security partnership through HashLock. Public users access DLMM trading, MEV protection, and credit-based AI chat with API integration across Discord and Telegram. Ignis Elite members receive exclusive access to automated 5-agent trading, 30% fee distribution, white-glove support, and priority features across Solana, Base, Arbitrum, and expanding chains.

Project Name: Umbrae

Project Type: DeFi

Logo:

U M Ǝ ɹ A ≡

Visualised Context:

Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Umbrae project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Umbrae Project Code Base
Platform	Ethereum / Solidity/Web apps & APIs
Penetration Test Date	August, 2025
Repository/Website URL/IP Tested	https://github.com/Iqnis-AI-Labs/umbrae/
GitHub Commit Hash (If applicable)	10afa45405c19d0c450dd3cedd80b3ba4e2c204a
Fix Commit Hash (If applicable)	50c5b2a1fc7488354c511fe62d7488eb1e4ba14b

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

6 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

6 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

Medium

[M-01] Umbrae - Transaction Fee Payments Are Not Bundled With Primary Actions

Description

The system executes the primary financial operation (e.g., a token swap) and the collection of the service fee as two separate, sequential transactions. The fee transaction is only initiated after the primary transaction has successfully completed. This creates a race condition where the primary action can succeed, but the subsequent fee collection might fail due to network issues, insufficient funds for the fee, or an expired blockhash. This was observed in `src/backend/services/swap/dex/jupiter.rs` and other files under `src/backend/services/swap/dex/` folder.

Impact

This sequential execution model introduces a significant risk of revenue loss. If the primary action (like a swap) is successfully completed but the fee transaction fails, the service will have provided its core functionality to the user without successfully collecting its fee. Over time, repeated failures of this nature can lead to a quantifiable loss of revenue for the platform.

Recommendation

To mitigate this risk, it is strongly recommended to bundle the fee payment and the primary action into a single, atomic transaction. All instructions—both for the main operation and the fee transfer—should be included in the same `VersionedTransaction`. This ensures that either the entire set of operations succeeds or it fails together, guaranteeing that a fee is collected if and only if the user's requested action is also successfully executed.

Status

Resolved

[M-02] Umbrae - Multiple unwrap/expect/panic paths in wallet and flows**Description**

Throughout the wallet and transaction processing flows in `/src/backend/services/wallet.rs`, there are multiple instances where `.unwrap()` and `.expect()` are used to handle `Option` and `Result` types. This includes operations like parsing public keys, unpacking token account data, creating transfer instructions, and looking up token decimals. Using these constructs assumes that the operations will always succeed, which is not a safe assumption when dealing with external data from the blockchain or APIs.

Impact

The use of `unwrap()` and `expect()` creates a significant denial-of-service (DoS) risk. If any of these operations fail—due to malformed data from an API, an unexpected account state on-chain, or an invalid user input—the entire service thread will panic and crash. This can disrupt all ongoing user operations, interrupt sensitive financial transactions mid-flow, and lead to an unreliable and unstable service.

Recommendation

It is recommended to replace all instances of `.unwrap()` and `.expect()` with robust error handling and propagation. Utilize typed errors to provide clear and actionable diagnostics. All inputs from external sources, including RPC responses and API data, should be validated, and the code should gracefully handle inconsistencies or failures without crashing the service.

Status

Resolved

[M-03] Umbrae - Unbounded Priority Fee Parameter in Withdrawals

Description

The withdrawal function in `/src/backend/services/withdraw.rs` accepts a `priority_fee` parameter from the user and directly uses it to set the compute unit price for the transaction via `ComputeBudgetInstruction::set_compute_unit_price`. There is no validation or upper limit imposed on the value of this parameter.

Impact

This lack of validation creates a financial security risk. A malicious actor or a user making a mistake could provide an extremely high value for the priority fee. This would cause the withdrawal transaction to consume an excessive and unintended amount of funds for the transaction fee, potentially draining a significant portion of the user's balance in a single operation.

Recommendation

To prevent this, the `priority_fee` parameter should be validated and clamped within a safe, pre-defined range. A server-side policy should enforce a maximum allowable priority fee (e.g., `MAX_PRIORITY_MICROLAMPORTS`). Any request with a fee exceeding this policy should be rejected. For greater security, consider making the fee policy entirely server-controlled, where the user-provided value is ignored in favor of a fee determined by the service based on current network conditions.

Status

Resolved

[M-04] Umbrae - DLMM Active Bin Calculation Uses Unnormalized Price in Fallback Path

Description

In the fallback logic for retrieving pool details (`get_pool_details_from_all_pools` in `/src/backend/api/dlmm_client.rs`), the `current_price` from the Meteora "all pools" API is used to calculate the `active_bin_id` without proper normalization. The calculation helper (`calculate_active_bin_id`) assumes the price is already adjusted for token decimals, which it is not. Furthermore, it uses `.round()` instead of `.floor()`, which can lead to off-by-one errors at bin boundaries.

Impact

This issue can lead to a significant misrepresentation of the pool's state to the user. When the fallback logic is triggered, the calculated active bin will be incorrect, especially when the two tokens in the pair have different numbers of decimals. This can mislead users by displaying incorrect price ranges and pool information, potentially causing them to make poor trading or liquidity provision decisions. While core actions that use the primary API path remain correct, frequent reliance on this fallback could severely degrade the user experience and trust in the platform's data.

Recommendation

The preferred solution is to avoid local calculation in the fallback path. Instead, query the transaction API's dedicated endpoints (like `/dlmm/get-active-bin`), which provide the authoritative active bin ID. If local calculation is unavoidable, it must be corrected to fetch the decimals for both tokens, normalize the price using the formula `p_norm = current_price * 10^(decX - decY)`, and then use `floor()` (with a small epsilon to handle floating-point inaccuracies) for the final bin ID calculation to align with on-chain semantics.

Status

Resolved

[M-05] Umbrae - Unchecked Unwraps on TX API Response can Panic

Description

When parsing the response from the transaction API in the `get_pool_details` function in `/src/backend/api/dlmm_client.rs`, the code directly calls `.unwrap()` on the result of the network request and subsequent JSON parsing. This assumes the API call will always succeed and that the returned JSON will always have the expected structure and fields.

Impact

This creates a reliability issue and a denial-of-service (DoS) vector. If the external API is temporarily unavailable, returns a non-200 status code, or sends a malformed or unexpected JSON payload, the `.unwrap()` calls will trigger a panic, crashing the service thread. This will interrupt any user operation that relies on this data, leading to a poor user experience and a brittle system.

Recommendation

To build a more resilient system, replace the `.unwrap()` calls with proper error handling. First, check the HTTP status code of the response to ensure it was successful. Then, safely parse the JSON, handling potential `Err` results. Finally, validate the presence and type of each field (e.g., `activeBin`) from the JSON payload before attempting to use it, returning a descriptive error to the caller instead of panicking.

Status

Resolved

[M-06] IAS-Portal-Holder - Webhook Endpoint Authentication Bypass

Description

The public-facing webhook endpoint in `api/src/api/webhook.js` at line 2 lacks any form of authentication or signature validation. This means that anyone with knowledge of the endpoint's URL and parameter structure can send crafted requests to it, which the system will process as if they were legitimate.

Impact

This vulnerability allows malicious actors to trigger unauthorized actions by sending fake webhook payloads. They could potentially manipulate user data, interfere with legitimate system operations, or cause data corruption. The lack of authentication opens a significant attack vector that could compromise the integrity and reliability of any feature that relies on these webhooks.

Recommendation

To secure the webhook endpoint, it is crucial to implement signature validation. Each incoming request should include a signature (e.g., an HMAC-SHA256 hash of the payload) in its headers, which the server can then verify using a shared secret. Additionally, consider IP whitelisting to only allow requests from known, trusted sources and apply rate limiting to prevent abuse of the endpoint.

Status

Resolved

Low

[L-01] Umbrae - Brute-Forceable Numeric PIN for AI Integration

Description

The system uses an integer (i32) PIN to authenticate users for AI integration features, as seen in the `get_user_by_wallet_and_pin` function in `/src/backend/db/users.rs`. This function grants access to a user's full record if the wallet address and PIN match. A numeric PIN, especially if not enforced to be of sufficient length, has very low entropy.

Impact

If an API endpoint exposes this wallet and PIN lookup mechanism, it becomes vulnerable to brute-force attacks. An attacker could rapidly guess the low-entropy numeric PINs for known wallet addresses, which could lead to unauthorized access to user data, linking of on-chain and off-chain identities, and potential abuse of the AI integration features.

Recommendation

It is recommended to move away from low-entropy numeric PINs for authentication. Replace them with high-entropy, cryptographically secure tokens or secrets. Implement strict rate-limiting and account lockout policies on authentication attempts. For a more secure flow, consider using time-limited one-time codes (OTPs) or challenge-response mechanisms involving cryptographic signatures instead of a static PIN.

Status

Resolved

[L-02] Umbrae - Umbrae Fee-mint not constrained to swap tokens

Description

The `ExecuteSwapParams` struct in `/src/backend/services/swap/transaction.rs` allows an `umbrae_fee_mint` to be specified, but there is no validation to ensure that this mint corresponds to either the input or output token of the swap. The DEX adapter functions (e.g., `src/backend/services/swap/dex/meteora.rs`) directly use this provided fee mint to initiate the fee collection transaction.

Impact

This lack of validation introduces both a business and an operational risk. A misconfigured or malicious call could specify a low-value or non-preferred token for the fee, leading to revenue loss. Operationally, if an incorrect mint is provided for which the system cannot find the correct number of decimals, the `umbrae_fee_mint` function defaults to 6 decimals. This can cause the `transfer_checked` instruction to fail, which, due to the non-atomic nature of the fee transfer, would result in the swap succeeding while the fee collection fails.

Recommendation

It is recommended to enforce an invariant in the `execute_swap` function to validate the `umbrae_fee_mint`. The function should reject any request where the fee mint is not one of the two tokens involved in the swap (input or output). Alternatively, maintain a server-side allowlist of acceptable fee tokens (e.g., SOL, USDC). Additionally, the `send_umbrae_fee` function should be strengthened to fail fast if it cannot look up the token decimals, rather than defaulting to a potentially incorrect value.

Status

Resolved

[L-03] IAS-Portal-Holder - Missing Input Validation in License Creation

Description

In the license creation endpoint in `/api/src/api/licenses.js`, user-provided inputs such as `broker` and `accountId` are used directly in a SQL INSERT statement without any prior validation or sanitization. This trusts that the user will provide well-formed and non-malicious data.

Impact

This lack of validation exposes the system to several risks. Invalid data could lead to database errors and data corruption, while maliciously crafted input could potentially lead to SQL injection attacks or other unexpected application behavior. The integrity of the licenses table is at risk, which could have downstream effects on any feature that relies on this data.

Recommendation

Implement strict input validation for all user-provided data before it is used in database operations. Use a schema-based validation library like Joi or Zod to define and enforce rules for each parameter, such as type, length, and format. All string inputs should be properly sanitized to prevent injection attacks and ensure data integrity.

Status

Resolved

QA

[Q-01] Umbrae - Plaintext Wallet Password Stored in Database

Description

The user's wallet password is being stored in plaintext in the database, as indicated by the `wallet_password: Option<String>` field in the `users` table in `/src/backend/db/users.rs`. Database operations directly read and write this sensitive field.

This is a critical security vulnerability. If the database is compromised, an attacker would gain immediate access to the decryption passwords for all user wallets. This would allow them to decrypt the stored wallet key pairs, leading to a complete compromise of user funds across the entire platform.

Recommendation

Do not store user passwords in a recoverable format. The password should only be used transiently to derive an encryption key. Use a strong, memory-hard Key Derivation Function (KDF) like Argon2id with a unique, per-user random salt. The derived key can be used for client-side encryption, or transiently on the server, but should never be stored. If password validation is required, store only the salted KDF hash of the password, which can be used for verification but not for decryption.

Status

Resolved

[Q-02] Umbrae - Weak Key Derivation for Wallet Encryption (No Salt, Single SHA-256)

Description

The wallet encryption key is derived by taking a single SHA-256 hash of the user's password in `/src/backend/services/wallet.rs`. This method does not use a salt(though it uses a random nonce) and lacks a sufficient work factor, making it computationally inexpensive to attack.

This weak key derivation scheme makes the encrypted wallet keys highly vulnerable to offline brute-force and dictionary attacks. If the encrypted key data is leaked from the database, an attacker can rapidly test common passwords to find a match. Since there is no salt, all users with the same password will have the same encryption key, allowing an attacker to crack multiple wallets at once.

Recommendation

Replace the current SHA-256 key derivation with a modern, memory-hard Key Derivation Function (KDF) such as Argon2id or scrypt. Each user's key derivation must incorporate a unique, randomly generated salt, which should be stored alongside the ciphertext. The KDF should be configured with high memory and time cost parameters to make brute-force attacks computationally infeasible.

Status

Resolved

[Q-03] IAS-Portal-Holder - Unnecessary Database Query for Empty Values

Description

The system was observed querying the database for `telegramId` and `discordId` for users in `/api/src/api/umbrae.js`, even in flows where these values are known to be null because they are intentionally not set during user creation. This results in unnecessary database read operations for data that is expected to be absent.

While not a direct security threat, this pattern of unnecessary database queries creates needless load on the database, reducing overall system efficiency and performance. It can also lead to confusion for developers maintaining the code, as the data flow becomes less clear, potentially causing issues in downstream processing or future feature development.

Recommendation

Refactor the data access logic to only query for optional fields like `telegramId` and `discordId` when they are explicitly needed for a given operation. Adopting a lazy-loading pattern for this type of non-essential user data would be more efficient. For a more structural fix, consider redesigning the database schema to separate core user data from optional social media links, which would clean up the data model.

Status

Resolved

[Q-04] IAS-Portal-Holder - Unused Parameter in License Creation

Description

During the license creation process in `/api/src/api/licenses.js` at line 94, the API endpoint extracts a `botName` parameter from the incoming request body. However, this parameter is never used in any subsequent logic or database operations, making it redundant.

This unused parameter represents dead code that can confuse developers and increase the maintenance burden. It suggests either incomplete or abandoned functionality, and

it unnecessarily expands the API's surface area, potentially creating a vector for future vulnerabilities if the handling of this parameter is changed insecurely.

Recommendation

If the `botName` parameter is not required for any current or planned functionality, it should be removed from the API endpoint and the corresponding processing logic to simplify the codebase. If it is intended for future use, this should be clearly documented with TODO comments, and the associated feature should be implemented. Ensure the API documentation is updated to reflect the actual parameters being used.

Status

Resolved

[Q-05] IAS-Portal-Holder - Confusing License Validation Logic

Description

The logic for validating a license holder's status in `/api/src/api/shadowCloneLicenses.js` at line 167 contains a confusing conditional statement (`if (!license.holderActive && license.holderActive !== false)`). This complex Boolean logic is hard to read and makes it difficult to understand the intended validation flow, especially concerning how null, undefined, and false values are handled.

Unclear and complex conditional logic increases the risk of bugs and unintended behavior. It could lead to legitimate users being denied access or, conversely, unauthorized users gaining access due to an edge case in the boolean evaluation that was not immediately obvious to the developer. It also makes the code harder to maintain and test.

Recommendation

Refactor the validation logic to be simple, explicit, and readable. Replace the complex condition with clear, separate checks for null or undefined values versus an explicit false value. This will make the code's intent clear and reduce the likelihood of introducing bugs. Adding comprehensive unit tests covering all possible validation scenarios is also recommended to ensure correctness.

Status

Resolved

[Q-06] IAS-Portal-Holder - Redundant Access Validation Logic**Description**

Access validation checks for Umbrae features are performed redundantly in multiple places within the `/api/src/api/umbrae.js` API file (e.g., line 125). The same validation logic is repeated for different endpoints (e.g., Telegram and Discord integration), which leads to code duplication and increased complexity.

This redundancy makes the codebase harder to maintain. If the validation logic needs to be updated, a developer must find and change every instance, which is error-prone and can easily lead to inconsistencies. This increases the risk of bugs where one code path is fixed but another is not, potentially leaving security holes.

Recommendation

Consolidate the redundant validation logic into a single, reusable function or middleware. This function should be called from all endpoints that require the access check. This approach follows the DRY (Don't Repeat Yourself) principle, making the code cleaner, easier to maintain, and ensuring that any future changes to the validation logic are applied consistently across the entire application.

Status

Resolved

Conclusion

After Hashlock's analysis, the Umbrae project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.